

SOURCE CODE DOCUMENT

SQUAT-TRACK: AI-Powered Squat Exercise Tracking System

COPYRIGHT REGISTRATION DOCUMENT

Application Type: Computer Software

Title of Work: SQUAT-TRACK

Author: Himanshu Sood

Date of Creation: February 21, 2025

Date of Publication: February 21, 2025

Repository: <https://github.com/himanshsood/SQUAT-TRACK> (<https://github.com/himanshsood/SQUAT-TRACK>)

SECTION 1: PROJECT OVERVIEW

SQUAT-TRACK is an original computer software application designed to track and count squat exercises in real-time using computer vision and machine learning technologies. The software consists of two primary components: a React-based frontend application and a Node.js backend server.

SECTION 2: ORIGINAL SOURCE CODE LISTING

2.1 Backend Core Logic (Node.js)

2.1.1 Main Server Entry Point (backend/index.js)

```

// SQUAT-TRACK Backend Server
// Original Code by Himanshu Sood
// Copyright © 2025 Himanshu Sood. All Rights Reserved.

const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');
const dotenv = require('dotenv');

// Configuration imports
const dbConfig = require('./config/db');
const authMiddleware = require('./middlewares/auth');

// Route imports
const userRoutes = require('./routes/userRoutes');
const workoutRoutes = require('./routes/workoutRoutes');

// Initialize Express application
const app = express();

// Middleware configuration
app.use(cors({
  origin: process.env.FRONTEND_URL || 'http://localhost:3000',
  credentials: true
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Database connection
mongoose.connect(dbConfig.mongoURI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('MongoDB Connected Successfully'))
.catch(err => console.error('Database Connection Error:', err));

// API Routes
app.use('/api/users', userRoutes);
app.use('/api/workouts', authMiddleware, workoutRoutes);

// Server initialization
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`SQUAT-TRACK Server running on port ${PORT}`);
});

```

2.1.2 Database Configuration (backend/config/db.js)

```
// Database Configuration
// Original Code by Himanshu Sood

module.exports = {
  mongoURI: process.env.MONGODB_URI || 'mongodb://localhost:27017/squattrack',
  jwtSecret: process.env.JWT_SECRET || 'squattrack_secret_key',
  jwtExpiration: '24h'
};
```

2.1.3 User Model (backend/models/User.js)

```
// User Model Schema
// Original Code by Himanshu Sood

const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const UserSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true
  },
  password: {
    type: String,
    required: true,
    minlength: 6
  },
  workoutHistory: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Workout'
  }],
  totalSquats: {
    type: Number,
    default: 0
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Password hashing middleware
UserSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Password verification method
UserSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

module.exports = mongoose.model('User', UserSchema);
```

2.1.4 Workout Model (backend/models/Workout.js)

```
// Workout Model Schema
// Original Code by Himanshu Sood

const mongoose = require('mongoose');

const WorkoutSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  squatCount: {
    type: Number,
    required: true,
    min: 0
  },
  duration: {
    type: Number, // in seconds
    required: true
  },
  calories: {
    type: Number,
    default: 0
  },
  formAccuracy: {
    type: Number, // percentage 0-100
    min: 0,
    max: 100
  },
  sessionDate: {
    type: Date,
    default: Date.now
  }
});

// Calculate calories burned
WorkoutSchema.pre('save', function(next) {
  // Average calories per squat = 0.32 kcal
  this.calories = Math.round(this.squatCount * 0.32);
  next();
});

module.exports = mongoose.model('Workout', WorkoutSchema);
```

2.1.5 Authentication Middleware (backend/middlewares/auth.js)

```

// JWT Authentication Middleware
// Original Code by Himanshu Sood

const jwt = require('jsonwebtoken');
const config = require('../config/db');

module.exports = function(req, res, next) {
  // Get token from header
  const token = req.header('Authorization')?.replace('Bearer ', '');

  // Check if no token
  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'No token, authorization denied'
    });
  }

  try {
    // Verify token
    const decoded = jwt.verify(token, config.jwtSecret);
    req.user = decoded.user;
    next();
  } catch (err) {
    res.status(401).json({
      success: false,
      message: 'Token is not valid'
    });
  }
};

```

2.1.6 User Routes (backend/routes/userRoutes.js)

```

// User Authentication Routes
// Original Code by Himanshu Sood

const express = require('express');
const router = express.Router();
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const config = require('../config/db');

// @route   POST /api/users/register
// @desc    Register new user
// @access  Public
router.post('/register', async (req, res) => {
  try {
    const { username, email, password } = req.body;

    // Check if user exists
    let existingUser = await User.findOne({
      $or: [{ email }, { username }]
    });

    if (existingUser) {
      return res.status(400).json({
        success: false,
        message: 'User already exists'
      });
    }

    // Create new user
    const user = new User({ username, email, password });
    await user.save();

    // Generate JWT token
    const payload = { user: { id: user.id } };
    const token = jwt.sign(payload, config.jwtSecret, {
      expiresIn: config.jwtExpiration
    });

    res.status(201).json({
      success: true,
      token,
      user: { id: user.id, username, email }
    });
  } catch (err) {
    console.error(err.message);
    res.status(500).json({
      success: false,
      message: 'Server error'
    });
  }
});

```

```

// @route   POST /api/users/login
// @desc    Authenticate user & get token
// @access  Public
router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    // Find user by email
    const user = await User.findOne({ email });
    if (!user) {
      return res.status(400).json({
        success: false,
        message: 'Invalid credentials'
      });
    }

    // Verify password
    const isMatch = await user.comparePassword(password);
    if (!isMatch) {
      return res.status(400).json({
        success: false,
        message: 'Invalid credentials'
      });
    }

    // Generate token
    const payload = { user: { id: user.id } };
    const token = jwt.sign(payload, config.jwtSecret, {
      expiresIn: config.jwtExpiration
    });

    res.json({
      success: true,
      token,
      user: {
        id: user.id,
        username: user.username,
        email: user.email,
        totalSquats: user.totalSquats
      }
    });
  } catch (err) {
    console.error(err.message);
    res.status(500).json({
      success: false,
      message: 'Server error'
    });
  }
});

module.exports = router;

```


2.1.7 Workout Routes (backend/routes/workoutRoutes.js)

```

// Workout Management Routes
// Original Code by Himanshu Sood

const express = require('express');
const router = express.Router();
const Workout = require('../models/Workout');
const User = require('../models/User');

// @route   POST /api/workouts
// @desc    Save new workout session
// @access  Private
router.post('/', async (req, res) => {
  try {
    const { squatCount, duration, formAccuracy } = req.body;

    const workout = new Workout({
      user: req.user.id,
      squatCount,
      duration,
      formAccuracy
    });

    await workout.save();

    // Update user's total squat count
    await User.findByIdAndUpdate(req.user.id, {
      $inc: { totalSquats: squatCount },
      $push: { workoutHistory: workout._id }
    });

    res.status(201).json({
      success: true,
      workout
    });
  } catch (err) {
    console.error(err.message);
    res.status(500).json({
      success: false,
      message: 'Server error'
    });
  }
});

// @route   GET /api/workouts
// @desc    Get user's workout history
// @access  Private
router.get('/', async (req, res) => {
  try {
    const workouts = await Workout.find({ user: req.user.id })
      .sort({ sessionDate: -1 })
      .limit(50);
  }
});

```

```

        res.json({
            success: true,
            count: workouts.length,
            workouts
        });
    } catch (err) {
        console.error(err.message);
        res.status(500).json({
            success: false,
            message: 'Server error'
        });
    }
});

// @route   GET /api/workouts/stats
// @desc    Get user's workout statistics
// @access  Private
router.get('/stats', async (req, res) => {
    try {
        const stats = await Workout.aggregate([
            { $match: { user: mongoose.Types.ObjectId(req.user.id) } },
            {
                $group: {
                    _id: null,
                    totalSquats: { $sum: '$squatCount' },
                    totalWorkouts: { $sum: 1 },
                    totalDuration: { $sum: '$duration' },
                    totalCalories: { $sum: '$calories' },
                    avgFormAccuracy: { $avg: '$formAccuracy' }
                }
            }
        ]);

        res.json({
            success: true,
            stats: stats[0] || {
                totalSquats: 0,
                totalWorkouts: 0,
                totalDuration: 0,
                totalCalories: 0,
                avgFormAccuracy: 0
            }
        });
    } catch (err) {
        console.error(err.message);
        res.status(500).json({
            success: false,
            message: 'Server error'
        });
    }
});

```

```
module.exports = router;
```

2.2 Frontend Core Logic (React.js)

2.2.1 Main Application Component (frontend/src/App.js)

```
// SQUAT-TRACK Main Application
// Original Code by Himanshu Sood
// Copyright © 2025 Himanshu Sood. All Rights Reserved.

import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import { WorkoutProvider } from '../context/WorkoutContext';
import Navbar from '../components/Navbar';
import Home from '../pages/Home';
import Dashboard from '../pages/Dashboard';
import Workout from '../pages/Workout';
import History from '../pages/History';
import Login from '../pages/Login';
import Register from '../pages/Register';
import PrivateRoute from '../components/PrivateRoute';
import './App.css';

function App() {
  return (
    <AuthProvider>
      <WorkoutProvider>
        <Router>
          <div className="app">
            <Navbar />
            <main className="main-content">
              <Routes>
                <Route path="/" element={<Home />} />
                <Route path="/login" element={<Login />} />
                <Route path="/register" element={<Register />} />
                <Route
                  path="/dashboard"
                  element={
                    <PrivateRoute>
                      <Dashboard />
                    </PrivateRoute>
                  }
                />
                <Route
                  path="/workout"
                  element={
                    <PrivateRoute>
                      <Workout />
                    </PrivateRoute>
                  }
                />
                <Route
                  path="/history"
                  element={
                    <PrivateRoute>
                      <History />
                    </PrivateRoute>
                  }
                />
              </Routes>
            </main>
          </div>
        </Router>
      </WorkoutProvider>
    </AuthProvider>
  );
}
```

```
        }  
      />  
    </Routes>  
  </main>  
</div>  
</Router>  
</WorkoutProvider>  
</AuthProvider>  
);  
}  
  
export default App;
```

2.2.2 Squat Counter Algorithm (frontend/src/utls/SquatCounter.js)

```

// Squat Detection and Counting Algorithm
// Original Code by Himanshu Sood
// Core Algorithm for Exercise Detection

class SquatCounter {
  constructor() {
    this.count = 0;
    this.stage = 'up'; // 'up' or 'down'
    this.angleThresholdDown = 90; // Knee angle for squat down position
    this.angleThresholdUp = 160; // Knee angle for standing position
    this.lastAngle = 180;
    this.minSquatDuration = 500; // Minimum ms between squats
    this.lastSquatTime = 0;
  }

  // Calculate angle between three points
  calculateAngle(pointA, pointB, pointC) {
    const radians = Math.atan2(pointC.y - pointB.y, pointC.x - pointB.x) -
      Math.atan2(pointA.y - pointB.y, pointA.x - pointB.x);
    let angle = Math.abs(radians * 180.0 / Math.PI);

    if (angle > 180.0) {
      angle = 360 - angle;
    }

    return angle;
  }

  // Process pose landmarks and detect squat
  processPose(landmarks) {
    if (!landmarks || landmarks.length < 33) {
      return { count: this.count, stage: this.stage, feedback: null };
    }

    // Get key points (using MediaPipe Pose landmark indices)
    const hip = landmarks[24]; // Right hip
    const knee = landmarks[26]; // Right knee
    const ankle = landmarks[28]; // Right ankle

    // Calculate knee angle
    const kneeAngle = this.calculateAngle(hip, knee, ankle);

    let feedback = null;
    const currentTime = Date.now();

    // Detect squat phases
    if (kneeAngle < this.angleThresholdDown && this.stage === 'up') {
      // User went down
      this.stage = 'down';
      feedback = 'Good depth! Now stand up.';
    } else if (kneeAngle > this.angleThresholdUp && this.stage === 'down') {
      // User stood up - count the squat

```

```

        if (currentTime - this.lastSquatTime > this.minSquatDuration) {
            this.stage = 'up';
            this.count++;
            this.lastSquatTime = currentTime;
            feedback = `Squat ${this.count} completed!`;
        }
    }

    // Form feedback
    if (kneeAngle < 70) {
        feedback = 'Too deep! Watch your form.';
    }

    this.lastAngle = kneeAngle;

    return {
        count: this.count,
        stage: this.stage,
        angle: kneeAngle,
        feedback: feedback
    };
}

// Reset counter
reset() {
    this.count = 0;
    this.stage = 'up';
    this.lastAngle = 180;
    this.lastSquatTime = 0;
}

// Get current count
getCount() {
    return this.count;
}
}

export default SquatCounter;

```

2.2.3 Pose Detection Component (frontend/src/components/PoseDetector.js)


```

// Real-time Pose Detection Component
// Original Code by Himanshu Sood
// Uses TensorFlow.js and MediaPipe for pose estimation

import React, { useRef, useEffect, useState, useCallback } from 'react';
import * as poseDetection from '@tensorflow-models/pose-detection';
import * as tf from '@tensorflow/tfjs';
import Webcam from 'react-webcam';
import SquatCounter from '../utils/SquatCounter';
import './PoseDetector.css';

const PoseDetector = ({ onSquatCount, onSessionEnd }) => {
  const webcamRef = useRef(null);
  const canvasRef = useRef(null);
  const [detector, setDetector] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [squatData, setSquatData] = useState({ count: 0, feedback: '' });
  const squatCounterRef = useRef(new SquatCounter());
  const animationRef = useRef(null);

  // Initialize pose detector
  useEffect(() => {
    const initDetector = async () => {
      await tf.ready();

      const detectorConfig = {
        modelType: poseDetection.movenet.modelType.SINGLEPOSE_LIGHTNING,
        enableSmoothing: true
      };

      const detector = await poseDetection.createDetector(
        poseDetection.SupportedModels.MoveNet,
        detectorConfig
      );

      setDetector(detector);
      setIsLoading(false);
    };

    initDetector();

    return () => {
      if (animationRef.current) {
        cancelAnimationFrame(animationRef.current);
      }
    };
  }, []);

  // Draw pose skeleton on canvas
  const drawPose = useCallback((pose, ctx, width, height) => {
    const keypoints = pose.keypoints;

```

```

// Draw keypoints
keypoints.forEach(keypoint => {
  if (keypoint.score > 0.3) {
    ctx.beginPath();
    ctx.arc(keypoint.x, keypoint.y, 5, 0, 2 * Math.PI);
    ctx.fillStyle = '#00ff00';
    ctx.fill();
  }
});

// Draw skeleton connections
const connections = [
  [11, 13], [13, 15], // Left arm
  [12, 14], [14, 16], // Right arm
  [11, 12], // Shoulders
  [11, 23], [12, 24], // Torso
  [23, 24], // Hips
  [23, 25], [25, 27], // Left leg
  [24, 26], [26, 28] // Right leg
];

ctx.strokeStyle = '#00ff00';
ctx.lineWidth = 2;

connections.forEach(([start, end]) => {
  const startPoint = keypoints[start];
  const endPoint = keypoints[end];

  if (startPoint.score > 0.3 && endPoint.score > 0.3) {
    ctx.beginPath();
    ctx.moveTo(startPoint.x, startPoint.y);
    ctx.lineTo(endPoint.x, endPoint.y);
    ctx.stroke();
  }
});
}, []);

// Main detection loop
const detectPose = useCallback(async () => {
  if (!detector || !webcamRef.current || !canvasRef.current) {
    animationRef.current = requestAnimationFrame(detectPose);
    return;
  }

  const video = webcamRef.current.video;
  if (video.readyState !== 4) {
    animationRef.current = requestAnimationFrame(detectPose);
    return;
  }

  const canvas = canvasRef.current;
  const ctx = canvas.getContext('2d');

```

```

    canvas.width = video.videoWidth;
    canvas.height = video.videoHeight;

    try {
        // Detect poses
        const poses = await detector.estimatePoses(video);

        // Clear and draw video frame
        ctx.clearRect(0, 0, canvas.width, canvas.height);
        ctx.drawImage(video, 0, 0, canvas.width, canvas.height);

        if (poses.length > 0) {
            const pose = poses[0];

            // Draw skeleton
            drawPose(pose, ctx, canvas.width, canvas.height);

            // Process squat detection
            const result = squatCounterRef.current.processPose(pose.keypoints);

            if (result.count !== squatData.count) {
                setSquatData({
                    count: result.count,
                    feedback: result.feedback || ''
                });

                if (onSquatCount) {
                    onSquatCount(result.count);
                }
            }
        }
    } catch (error) {
        console.error('Pose detection error:', error);
    }

    animationRef.current = requestAnimationFrame(detectPose);
}, [detector, drawPose, onSquatCount, squatData.count]);

// Start detection when detector is ready
useEffect(() => {
    if (detector && !isLoading) {
        detectPose();
    }
}, [detector, isLoading, detectPose]);

if (isLoading) {
    return (
        <div className="pose-detector-loading">
            <div className="spinner"></div>
            <p>Loading AI Model...</p>
        </div>
    );
}

```

```

    );
}

return (
  <div className="pose-detector">
    <div className="video-container">
      <Webcam
        ref={webcamRef}
        className="webcam"
        mirrored={true}
        videoConstraints={{
          width: 640,
          height: 480,
          facingMode: 'user'
        }}
      />
      <canvas ref={canvasRef} className="overlay-canvas" />
    </div>
    <div className="squat-counter-display">
      <h2>Squats: {squatData.count}</h2>
      {squatData.feedback && (
        <p className="feedback">{squatData.feedback}</p>
      )}
    </div>
  </div>
);
};

export default PoseDetector;

```

2.2.4 Authentication Context (frontend/src/context/AuthContext.js)

```

// Authentication Context Provider
// Original Code by Himanshu Sood

import React, { createContext, useContext, useState, useEffect } from 'react';
import axios from 'axios';

const AuthContext = createContext();

export const useAuth = () => {
  return useContext(AuthContext);
};

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token'));
  const [loading, setLoading] = useState(true);

  const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

  // Set axios default header
  useEffect(() => {
    if (token) {
      axios.defaults.headers.common['Authorization'] = `Bearer ${token}`;
    } else {
      delete axios.defaults.headers.common['Authorization'];
    }
  }, [token]);

  // Load user on mount
  useEffect(() => {
    const loadUser = async () => {
      if (token) {
        try {
          const response = await axios.get(`${API_URL}/users/me`);
          setUser(response.data.user);
        } catch (error) {
          console.error('Failed to load user:', error);
          logout();
        }
      }
      setLoading(false);
    };

    loadUser();
  }, [token, API_URL]);

  const login = async (email, password) => {
    try {
      const response = await axios.post(`${API_URL}/users/login`, {
        email,
        password
      });
    }
  };

```

```

        const { token: newToken, user: userData } = response.data;

        localStorage.setItem('token', newToken);
        setToken(newToken);
        setUser(userData);

        return { success: true };
    } catch (error) {
        return {
            success: false,
            message: error.response?.data?.message || 'Login failed'
        };
    }
};

const register = async (username, email, password) => {
    try {
        const response = await axios.post(`${API_URL}/users/register`, {
            username,
            email,
            password
        });

        const { token: newToken, user: userData } = response.data;

        localStorage.setItem('token', newToken);
        setToken(newToken);
        setUser(userData);

        return { success: true };
    } catch (error) {
        return {
            success: false,
            message: error.response?.data?.message || 'Registration failed'
        };
    }
};

const logout = () => {
    localStorage.removeItem('token');
    setToken(null);
    setUser(null);
};

const value = {
    user,
    token,
    loading,
    login,
    register,
    logout,

```

```
    isAuthenticated: !!token
  };

  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
};
```

2.2.5 Workout Context (frontend/src/context/WorkoutContext.js)

```

// Workout State Management Context
// Original Code by Himanshu Sood

import React, { createContext, useContext, useState, useCallback } from 'react';
import axios from 'axios';
import { useAuth } from './AuthContext';

const WorkoutContext = createContext();

export const useWorkout = () => {
  return useContext(WorkoutContext);
};

export const WorkoutProvider = ({ children }) => {
  const { token } = useAuth();
  const [currentSession, setCurrentSession] = useState(null);
  const [workoutHistory, setWorkoutHistory] = useState([]);
  const [stats, setStats] = useState(null);
  const [loading, setLoading] = useState(false);

  const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

  // Start new workout session
  const startSession = useCallback(() => {
    setCurrentSession({
      startTime: Date.now(),
      squatCount: 0,
      formAccuracy: []
    });
  }, []);

  // Update squat count during session
  const updateSquatCount = useCallback((count) => {
    setCurrentSession(prev => ({
      ...prev,
      squatCount: count
    }));
  }, []);

  // End and save workout session
  const endSession = useCallback(async () => {
    if (!currentSession) return { success: false };

    const duration = Math.round((Date.now() - currentSession.startTime) / 1000);
    const avgFormAccuracy = currentSession.formAccuracy.length > 0
      ? currentSession.formAccuracy.reduce((a, b) => a + b, 0) / currentSession.formAccuracy.length
      : 85;

    try {
      setLoading(true);
      const response = await axios.post(`${API_URL}/workouts`, {
        squatCount: currentSession.squatCount,

```



```

        duration: duration,
        formAccuracy: Math.round(avgFormAccuracy)
    });

    setCurrentSession(null);
    setLoading(false);

    return {
        success: true,
        workout: response.data.workout
    };
} catch (error) {
    setLoading(false);
    return {
        success: false,
        message: error.response?.data?.message || 'Failed to save workout'
    };
}
}, [currentSession, API_URL]);

// Fetch workout history
const fetchHistory = useCallback(async () => {
    try {
        setLoading(true);
        const response = await axios.get(`${API_URL}/workouts`);
        setWorkoutHistory(response.data.workouts);
        setLoading(false);
    } catch (error) {
        console.error('Failed to fetch history:', error);
        setLoading(false);
    }
}, [API_URL]);

// Fetch user statistics
const fetchStats = useCallback(async () => {
    try {
        const response = await axios.get(`${API_URL}/workouts/stats`);
        setStats(response.data.stats);
    } catch (error) {
        console.error('Failed to fetch stats:', error);
    }
}, [API_URL]);

const value = {
    currentSession,
    workoutHistory,
    stats,
    loading,
    startSession,
    updateSquatCount,
    endSession,
    fetchHistory,

```

```
        fetchStats
    };

    return (
        <WorkoutContext.Provider value={value}>
            {children}
        </WorkoutContext.Provider>
    );
};
```

SECTION 3: CUSTOM STYLES (CSS)

```
/* SQUAT-TRACK Custom Styles
   Original CSS by Himanshu Sood */

/* Global Styles */
:root {
  --primary-color: #4f46e5;
  --secondary-color: #818cf8;
  --success-color: #10b981;
  --warning-color: #f59e0b;
  --error-color: #ef4444;
  --background-dark: #1f2937;
  --text-primary: #f9fafb;
  --text-secondary: #9ca3af;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', -apple-system, sans-serif;
  background: linear-gradient(135deg, #1f2937 0%, #111827 100%);
  color: var(--text-primary);
  min-height: 100vh;
}

.app {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

/* Pose Detector Styles */
.pose-detector {
  position: relative;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 20px;
}

.video-container {
  position: relative;
  border-radius: 16px;
  overflow: hidden;
  box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);
}

.webcam {
  display: block;
```

```
        border-radius: 16px;
    }

.overlay-canvas {
    position: absolute;
    top: 0;
    left: 0;
    border-radius: 16px;
}

.squat-counter-display {
    margin-top: 24px;
    text-align: center;
    background: rgba(79, 70, 229, 0.2);
    padding: 24px 48px;
    border-radius: 16px;
    backdrop-filter: blur(10px);
}

.squat-counter-display h2 {
    font-size: 3rem;
    font-weight: 700;
    color: var(--success-color);
}

.squat-counter-display .feedback {
    margin-top: 12px;
    font-size: 1.25rem;
    color: var(--secondary-color);
}

/* Loading Spinner */
.pose-detector-loading {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    min-height: 400px;
}

.spinner {
    width: 60px;
    height: 60px;
    border: 4px solid var(--secondary-color);
    border-top-color: var(--primary-color);
    border-radius: 50%;
    animation: spin 1s linear infinite;
}

@keyframes spin {
    to { transform: rotate(360deg); }
}
```

SECTION 4: DECLARATION

I, **Himanshu Sood**, hereby declare that:

1. The source code listed in this document is my original work.
2. I am the sole author and copyright holder of this software.
3. This code has not been copied from any other source or publication.
4. All third-party libraries (React, TensorFlow.js, Express, MongoDB) are used under their respective open-source licenses and are NOT claimed as my original work.
5. Only the application logic, algorithms, and custom code shown above constitute my copyrightable work.

Author Signature: Himansh Sood

END OF SOURCE CODE DOCUMENT